

Progetto Apollo (ingegneria del software)

Alessandro Lemme, Alessandro Lucchese

Università degli studi di Brescia

25 agosto 2026

1 Pattern GRASP

- Information Expert
- Creator
- Controller
- Pure Fabrication
- Polimorphism
- Legge di Demetra

2 Pattern SOLID

- SingleResponsibility
- Open Closed
- Interface Segregation Principle
- Dependency inversion
- Principio di separazione modello-vista

3 Gang og Four

- Simple Factory
- Strategy
- MVC

Qual è il principio base nell'assegnazione di responsabilità agli oggetti?

I vari engine hanno accesso a (parte) del database e soddisfano le richieste prendendo i dati a loro strettamente necessari e formulando una risposta - dunque la responsabilità di formulare una risposta è attribuita alla classe che ha la possibilità di reperire i dati necessari.

Chi dovrebbe essere responsabile per la creazione di una nuova istanza di una classe?

`Server` è responsabile della creazione degli engine, contiene i dati della richiesta - quindi possiede i dati per inizializzare l'oggetto che implementa `Engine` - ed in base a questo crea l'oggetto che possa formulare la risposta. Allo stesso modo `Client` crea la classe che implementa `RequestInterface` che corrisponde alla richiesta specifica: lo strato UI passa i dati necessari per inizializzarla e lui crea l'oggetto che permetterà poi di creare la stringa JSON.

Qual è il primo oggetto oltre lo strato UI che riceve e coordina un'operazione di sistema?

Qui creator e controller vanno a braccetto, lo strato UI chiama Client che crea l'oggetto corrispondente alla richiesta dell'utente. Allo stesso modo se per operazione di sistema si intende una modifica dei dati (e non il semplice invio della richiesta con un socket) allora Server possiede e coordina la creazione degli oggetti che effettueranno queste modifiche.

Come assegnare le responsabilità quando usando Information Expert si violano Low Coupling e High Cohesion?

Tutte le classi del pacchetto Engine hanno l'unica cosa in comune di agire sul database: mentre, essendo simil RESTful API (invece che POST si usa Set*, PUT diventa Edit*, GET e DELETE sono Get* e Delete*) al posto di esser divise per tipologia di richiesta sono tutte nello stesso pacchetto ordinate alfabeticamente di modo che il sistemista possa al bisogno alterare le query modificando un solo pacchetto (seguendo anche il principio del minimo privilegio, così facendo al sistemista può essere revocato il permesso di modifica al di fuori del suo pacchetto di competenza).

Come gestire responsabilità alternative che variano sulla base del tipo (la classe)?

Gli oggetti che implementano l'interfaccia `EngineInterface` sono un esempio di polimorfismo: implementano la funzione `handleRequest()` rendendo facile la creazione di un nuovo engine senza influenzare gli engine già esistenti o il modo in cui il server chiama la risposta dell'oggetto.

Come progettare uno o più oggetti in modo tale che l'instabilità di questi oggetti non abbia un impatto indesiderato su altri oggetti?

Ogni oggetto parla solo con il suo superiore/sottoposto: un engine parla solo con ServerAPI, ServerAPI solo o con gli Engine o con Server: non avviene mai uno scavalcamiento. Simile lato client.

Ogni oggetto ha una ed una sola responsabilità, gli Engine di comporre una risposta specifica alla loro funzione, ServerAPI di creare gli engine e carpirne la risposta sulla base della richiesta fatta da Server, Server di interrogare ServerAPI e rispondere all'IP che ha mandato la richiesta.

Per estendere le funzioni del server bisogna:

- Creare un oggetto in `Request` che implementi `RequestInterface` e prenda i dati necessari a formulare la richiesta.
- Creare un metodo in `Client` che prenda i dati e costruisca l'oggetto `Request`.
- Creare un oggetto in `Engine` che implementi `EngineInterface` e soddisfi la richiesta.
- Creare un oggetto in `Reply` che implementi `ReplyInterface` che contenga i dati richiesti.

Interface Segregation Principle

Ogni classe qualora implementasse un'interfaccia ne implementa soltanto una

Dependency inversion

Ogni dipendenza è rispetto ad una interfaccia (Engine, AuthenticatedEngine, Thread, ...)

I dati seguono una pipeline fortemente compartimentata - anche dovuto al modello architetturale Fat Client - l'interfaccia grafica dipende da una classe sottostante responsabile della creazione delle richieste (un oggetto JSON), la quale manda la richiesta al server che risponderà con un JSON che verrà "spacchettato" dal Client e che l'interfaccia grafica mostrerà all'utente.

Chi crea i vari Engine per risolvere la domanda del Client?

- **Client:** è il Client dell'utente, che chiama (indirettamente) i metodi di Server
- **Factory:** è Server che crea gli engine a seconda del tipo di richiesta del client.
- **Abstract:** è EngineInterface, implementata dai vari engine.
- **Product:** Sono i vari [Delete,Edit,Get,Set]*Engine.

- **Strategy:** Engine è l'interfaccia a cui tutti i contenitori di algoritmi devono rifarsi per essere creati ed invocati da Server.
- **Context:** EnginePureFabricator è il context - contiene un riferimento ad un Engine, EngineInterface è l'oggetto strategy, dichiara un'interfaccia comune agli algoritmi supportati mentre i vari *Engine sono i concrete strategy dato che implementano l'algoritmo usando strategy.
- **ConcreteStrategy:** i vari engine contengono l'algoritmo per rispondere alle richieste.

- **Model:** rappresenta i dati e il comportamento dell'applicazione, ovvero gli Engine e la loro politica interna di gestione dei dati.
- **View:** rappresenta il modo in cui l'utente interagisce con l'applicazione, ovvero la classe Client che effettua una chiamata tramite NetworkClient sulla porta 8000.
- **Controller:** trasforma le interazione dell'utente della View in richieste al Model, ovvero tutta l'interfaccia grafica che costruisce la richiesta pezzo per pezzo e la invia al sistema sottostante.

Quando viene effettuata una chiamata al database esistono poi degli oggetti - tutti quelli del package `Communication/DatabaseObjects` - che agiscono in modo coeso e sensato

Il database SQL di java.sql agisce da proxy, assieme ad H2

- remoto: qualora il DB fosse altrove l'interfaccia java.sql gestisce la connessione
- virtuale: H2 che implementa java.sql gestisce il caricamento in memoria dei dati del database
- protezione: l'accesso al database non è totalmente libero, serve una autenticazione
- intelligente: "consente l'esecuzione di attività aggiuntive quando si accede al soggetto" - ovvero le query

I test eseguiti su `GetPersonalDataEngine` sono tre per i due possibili metodi che potrebbero venir chiamati.

- `testPetitionerCanLogIn()` controlla solo che l'utente `Lancillotto.Benacense.99` possa connettersi con le sue credenziali (corrette).
- `testProcessWithConnection()` controlla che l'utente ritornato contenga tutti i dati che dovrebbe.
- `testProcessWithConnectionShouldFail()` controlla che un utente inesistente non possa accedere a dati del database.